



uProf User Guide (57368)

Performance Characterization Using AMDuProfPcm

4. Performance Characterization Using AMDuProfPcm

Performance Modelling using AMDuProfPcm

5. Performance Modelling using AMDuProfPcm

Performance Characterization Using AMDuProfPcm

4. Performance Characterization Using AMDuProfPcm

4.1. Overview

The System Analysis tool, AMDuProfPcm, helps you track, analyze, and understand performance metrics for AMD processors. It collects metrics from hardware performance monitoring counters and operating system interfaces, and displays them as time-series plots, graphs, or CSV reports. The tool also supports basic performance modeling using roofline plots.

4.2. Performance Monitoring Counters (PMC)

Performance Monitoring Counters (PMC) are hardware resources capable of counting events occurring inside the processor. To count a specific event, a counter is configured or loaded with a unique value referred to as an event configuration.

PMC is categorized as follows:

- **Core PMC:** Every logical core has a set of performance monitoring counters referred to as Core PMCs capable of counting events inside the core. For a list of all the supported events for Core PMCs, refer to [Core Performance Monitor Counters in the Processor Programming Reference](#), and search for the specific PPR.
- **L3 PMC (Uncore):** Every L3 cache has a set of performance monitoring counters referred to as L3 PMCs capable of counting events inside the L3 cache. For a list of all the supported events for L3 PMCs, see the section L3 Cache PMC Events in the [Processor Programming Reference](#), and search for the specific PPR. Metrics listed under option `-m` such as `l3` are based on L3 PMC events and are referred to as L3 metrics.
- **DF (Data Fabric) PMC (Uncore):** Every socket has a set of performance monitoring counters referred to as DF PMCs, capable of counting events inside the Data Fabric. For a list of all the supported events for DF PMCs, see the section Data Fabric Performance Monitor Events in the [Processor Programming Reference](#), and search for the specific PPR. Metrics listed under option `-m` such as `memory`, `xgmi`, `pcie`, `dma`, `ccm_bw` are based on DF PMC events and are referred to as DF metrics.

4.3. Key Features

4.3.1. Multiplexing

Multiplexing is required when the number of events to be monitored exceeds the number of available PMCs. Event configurations are grouped together to ensure metrics, which are computed using multiple events, are valid. Each group is configured or loaded onto the PMCs at a time in a round robin fashion. Each group resides on the PMCs for a duration before being replaced by the next group. For the duration an event group does not reside on the PMCs (since PMCs are occupied by other groups), the counts for those events are extrapolated from the last known measurement or sample. Due to this, multiplexing usually introduces noise in the collected data.

- **Multiplexing Interval:** The interval for which an event group resides on PMCs during multiplexing. Multiplexing interval can be set using `-t` option and should be kept as low as possible for data accuracy.
- **Logging Interval:** The interval after which a sample, containing the count values of all the events to be monitored, is logged. Logging interval can be set with `-l` option. When multiplexing, the logging interval must be a multiple of the product of the multiplexing interval and the number of event groups. AMDuProfPcm will adjust the logging interval to the closest value specified by the user while ensuring this condition is met.

4.3.2. Profiling Data Collection and Reporting

AMDuProfPcm collects profiling data in two modes:

- Perf Mode (Non root mode)(Only supported in Linux)
- MSR Mode (Root mode)

4.3.2.1. Perf Mode (Only available on Linux)

PMCs are managed by perf subsystem on Linux and AMDuProfPcm utilizes perf subsystem for accessing PMCs. Perf subsystem enables sharing of hardware resources (PMCs) which allows multiple instances of AMDuProfPcm and other profilers to run simultaneously. On Linux, it is the default mode of data collection.

Multiplexing interval in perf mode can be set by:

- Writing the value (in ms) to `/sys/devices/cpu/perf_event_mux_interval_ms` for Core PMCs.
- Writing the value (in ms) to `/sys/devices/amd_l3/perf_event_mux_interval_ms` for L3 PMCs.
- Writing the value (in ms) to `/sys/devices/amd_df/perf_event_mux_interval_ms` for DF PMCs.

Availability of `/sys/devices/amd_l3/perf_event_mux_interval_ms` and `/sys/devices/amd_l3/perf_event_mux_interval_ms` depend on kernel version and `amd_uncore` module.

4.3.2.2. MSR Mode

PMC are accessed by directly configuring PMC MSRs. Root privileges are required for this mode. Older Linux kernels might not support data collection with L3 PMCs and DF PMCs with Perf mode. In such scenarios you can use MSR mode.

To use MSR mode based on the various operating systems:

- **Linux:** PMC MSRs are accessed via msr kernel module. Use the `--msr` option to select this mode for data collection.
- **Windows:** PMC MSRs are accessed via ioctl call to AMDPowerProfiler driver and is the default mode for data collection.
- **FreeBSD:** PMC MSRs are accessed with the cpuctl module and is the default mode for data collection.

4.3.3. Data Collection and Reporting

AMDuProfPcm collects and reports the data in the following modes:

- **Timeseries Mode:** Profiling data is collected and reported periodically, and the logging interval can be specified with `-I` option. This is the default mode of data collection.
- **Cumulative Mode:** Profiling data is collected and accumulated over the profile run and is reported for the entire profile duration. Use `-C` option to select this mode.
- **Live Mode:** Profiling data is collected and printed on the console periodically. Use the `top` command to select this mode.

AMDuProfPcm reports data in the following formats:

- **HTML:** The HTML report features graphical representations of the monitored metrics, aiding users in visualizing the data. It includes several tabs that organize relevant metrics by PMC type and significance. The report presents a variety of plots and graphs, including time series plots, heatmaps, sunburst charts, and radar graphs. To generate the HTML report, use the `--html` option in the command.
- **CSV:** Profiling data is printed in comma separated value (.csv) format, which can be viewed using MS Excel or other similar applications or be used as input for custom scripts for further processing the data.
- **JSON:** The JSON report serves as an intermediate format for generating the final HTML report from the CSV report. It organizes performance data into several key sections: hierarchy, metadata, metric-groups, metrics, and sections.
 - The hierarchy section defines the structural relationship between different performance metrics.
 - The metadata section captures environment and system details relevant to the report.
 - The metric-groups section logically groups related metrics together.
 - The metrics section records the actual performance metrics collected during the observation period.
 - The sections section contains configuration for graph plotting based on the collected metrics.

The JSON report is automatically generated when the `--html` option is specified in the command.

4.3.4. Process Tracking (Only available in Perf mode)

AMDuProfPcm can profile multiple processes, collecting data exclusively for the attached process (designated with the `-p <pid,...>` or `-tid <tid,...>`) or application (specified via command). Other background processes running concurrently are excluded from data collection. This feature is only available in Perf mode, when gathering Core PMC metrics, and only supported with system-wide aggregation (using the `-A system` option).

Table 4.1 **Feature Support Matrix For AMD Zen Family Processors**[#]

Feature/AMD Zen Family	Zen2	Zen3	Zen4	Zen5
------------------------	------	------	------	------

Feature/AMD Zen Family	Zen2	Zen3	Zen4	Zen5
HTML Report	No	Yes	Yes	Yes
Virtualization	No	Yes	Yes	Yes
CSV Report	Yes	Yes	Yes	Yes
Roofline	Yes	Yes	Yes	Yes

4.4. Prerequisite(s)

4.4.2. Linux

In Perf Mode, AMD processors based on Zen4 and beyond require Linux kernel version 6.0 and greater for supporting collection of DF metrics. For data collection, set `/proc/sys/kernel/perf_event_paranoid` to *0 or lower*. For example: `echo 0 >> /proc/sys/kernel/perf_event_paranoid`.

In MSR Mode, AMDuProfPcm requires msr kernel module to be loaded to access the MSRs from userspace. After it is loaded, either write permission for `/dev/cpu*/msr` devices or root privileges are required.

The `AMDPcmSetCapability.sh` script has been newly added for PCM and after executing the script, you can run the option `--msr mode` without root privileges.

Note

Use a new terminal for profiling without root after setting the capability.

To load the msr module, use the command:

```
modprobe msr
```

To disable NMI watchdog, write 0 to `/proc/sys/kernel/nmi_watchdog`. For example: `echo 0 >> /proc/sys/kernel/nmi_watchdog`.

Roofline plotting script (`AMDuProfModelling.py`) requires python 3.x and python module matplotlib.

4.5. AMDuProfPcm Command Line Options

Here is a list of all command line options.

Table 4.2 AMDuProfPcm Command Line Options#

Option	Description
--------	-------------

Option	Description
-h	Displays help information on the console/terminal.
-m <metric,...>	Metrics to report. The supported metric groups and the corresponding metrics are Platform, OS, and Hypervisor specific. Run <code>./AMDuProfPcm -h</code> to get the list of supported metrics. The following metric groups are supported: Core

Option	Description
	• ipc: reports metrics such as CEF, Utilization, CPI, and IPC • fp: reports GFLOPS • dc: advanced caching metrics such as DC refills by source (supported only on AMD <i>Zen 3</i>, AMD <i>Zen 4</i>, and AMD <i>Zen 5</i> processors) • l1: L1 cache related metrics (DC access and IC Fetch miss ratio) • l2: L2D and L2I cache related access/hit/miss metrics • swpfdc: software prefetch data cache from various nodes and CCX (supported only on AMD <i>Zen 3</i>, AMD <i>Zen 4</i>, and AMD <i>Zen 5</i> processors) • hwpfdc: hardware prefetch data cache from various nodes and CCX (supported only on AMD <i>Zen 3</i>, AMD <i>Zen 4</i>, and AMD <i>Zen 5</i> processors) • pipeline_util: top-down metrics to visualize the bottlenecks in the CPU pipeline (supported only on AMD <i>Zen 4</i> and AMD <i>Zen 5</i> processors) • avx_imix: SSE/AVX instruction mix percentage • cache_miss: L1 Data and Instruction cache misses, L2 Data and Code Read misses • tlb: translation lookaside buffer misses L3

Option	Description
	• l3: L3 cache metrics like L3 Access, L3 Miss, and Average Miss latency DF Metrics • memory: approximate memory read and write bandwidths in GB/s for all the channels • pcie: PCIe bandwidth in GB/s (supported only on AMD <i>Zen 2</i>, AMD <i>Zen 4</i>, and AMD <i>Zen 5</i> processors) • xgmi: approximate xGMI outbound databytes in GB/s for all the remote links • dma: DMA bandwidth in GB/s (supported only on AMD <i>Zen 4</i> and AMD <i>Zen 5</i> processors) • ccm_bw: Core complex inbound and outbound bandwidth in GB/s (supported only on AMD <i>Zen 4</i> processors and later) • cxl: Compute Express Link bandwidth in GB/s (supported only on AMD <i>Zen 5</i> processors and later)
-c <core ccx l3 numa package>=<n>	Collect from the specified <code>core</code> <code>ccx</code> <code>ccd</code> <code>package</code>. The default is <code>core=0</code>. If ccx or l3 is specified:

Option	Description
	• Core events will be collected from all the cores of this ccx. • l3 event will be collected for ccx this core belongs to. • df events will be collected for the package this core belongs to. If package is specified: • Core events will be collected for all the cores of this package. • l3 events will be collected for all the ccxs of this package. • df events will be collected for this package. If numa is specified: • Core events will be collected for all the cores of this numa. • l3 events will be collected for all the ccxs of this numa. • df events will be collected for this numa.
-a	Collect from all the cores. Note Options -c and -a cannot be used together.
-C	Prints the cumulative data at the end of the profile duration. Else, all the samples will be reported as timeseries data.
-A <system, package, ccd, ccx, core>	Prints aggregated metrics at various component levels. The following

Option	Description
	granularities are supported: • system: samples from all the cores in the system will be aggregated. • package: samples from all the cores in the package will be aggregated and reported for all the packages available in the system; applicable for multi-package systems. • ccd: samples from all the cores in CCD will be aggregated and reported for all the CCDs. • ccx: samples from all the cores in CCX will be aggregated and reported for all the CCXs. Note CCX is only applicable to Core and L3 metrics. • core: samples from all the logical cores on which samples are collected will be reported without aggregation. Note ◦ Option <code>-a</code> should be used along with this option to collect samples from all the cores. ◦ Comma separated list of components can be specified.
-I <ms>	Print the metrics at regular intervals. By default, it is enabled with an interval of 1000 ms.

Option	Description
<code>-i <config file></code>	User defined XML config file that specifies <i>Core/L3/DF</i> counters to monitor. Refer sample files in the <i><install-dir>/bin/Data/Config/</i> directory for the format. Note - Options <code>-i</code> and <code>-m</code> cannot be used together. - If option <code>-i</code> is used, all the events mentioned in the user defined config file will be collected.
<code>--start-delay <n></code>	Start profiling after the specified duration (<i>n</i> in milliseconds).
<code>--wait-for-signal</code>	Wait for <i>SIGUSR1</i> to start profiling. Use <i>SIGINT/SIGTERM</i> to terminate. Note Supported only on Linux.
<code>-u, --os-user <n></code>	Override events to collect for kernel/user-only mode. Available options are: 0 - Count no events. 1 - Count user events. 2 - Count OS events. 3 - Count all events. Note Applicable only for Core metrics.
<code>--tid <tid,...></code>	Profile existing threads. Thread IDs are separated by comma. Note

Option	Description
	Available only for Core metrics. Not applicable with <code>--msr</code> option.
<code>-d <seconds></code>	Profile duration to run.
<code>-t <multiplex interval in ms></code>	The interval in which pmc count values will be read, the minimum is 100 ms.
<code>-o <output file></code>	The output file name, it is in CSV format.
<code>-P <n></code>	Sets precision of the metrics reported, the default value is 2.
<code>-q</code>	Hide CPU topology section in the output report.
<code>-r</code>	Force resets the MSRs.
<code>-k</code>	Prefixes <code>pkg</code> in package level counters.
<code>-s</code>	Displays time stamp in the time series report.
<code>-l</code>	Lists the supported raw PMC events.
<code>-z <pmc-event></code>	Prints the name, description, and available unit masks for the event.
<code>-w <dir></code>	Specifies the working directory. The default will be the path of the launched application.
<code>-n</code>	Print cpu topology.
<code>-v</code>	Print version.
<code>-X</code>	Collect data using perf subsystem without root privileges. This option

Option	Description
	is enabled by default and will be deprecated soon. Note This is only supported on Linux.
<code>-p <process ID></code>	Specify the target process ID to monitor. Note This is only supported with the option <code>-X</code> on Linux.
<code>-f <util:<n>></code>	Filter the roofline data based on the utilization. For example, <code>-f util:90</code> will filter all data points with less than 90% utilization. Note This is applicable only with the roofline command.
<code>--html</code>	Collect data and generate HTML report.
<code>--msr</code>	Collect data using MSR mode. Requires root privilege.
<code>--percentile</code>	Generate custom percentile in html report. Default is 95th Percentile.
<code>--collect-xgmi</code>	Collect xgmi data.
<code>--collect-pcie</code>	Collect pcie data.
<code>--collect-power</code>	Collect power data.
<code>--per-core</code>	Prints metrics at core level (no aggregation).
<code>--per-die</code>	Prints aggregated metrics at die level.

Option	Description
<code>--per-socket</code>	Prints aggregated metrics at socket level.
<code>--report-roofline</code>	Report roofline data under the 'profile' command.
<code>--no-aggr</code>	Prints non-aggregated metrics data: • Core metrics will be reported at core level. • l3 metrics will be reported at ccx level. • df metrics will be printed at package/aid level.
<code>--collect-clk</code>	Collect clock data.
<code>--collect-guest</code>	<u>Count only the guest events.</u> Note This is applicable only in the host when hypervisor is enabled.
<code>--collect-host</code>	Count only the host events. (Default behavior is to collect host and guest data). <u>Note</u> This is applicable only in the host when hypervisor is enabled.
<code>--read-smbios</code>	Read memory speed and total memory channels from SMBIOS. <u>Note</u> This is applicable only only for roofline command.
<code>--show-used-ccx</code>	Show only the CCXs where the target application ran in the heatmap's L3 Cache section in the html report.

Option	Description
	Note Not applicable with <code>--msr</code> option.
<code>-O <output dir path></code>	Path to create the output directory.

4.5.1. Commands

Here is a list of all the commands.

Table 4.3 **AMDuProfPcm Commands#**

Command	Description
<code>hreport</code>	Create html report from csv report.
<code>compare</code>	Compare two collected sessions and create an html comparison report.
<code>roofline</code>	Collects data required for generating roofline model.
<code>top</code>	Collect and report real time timeseries data in a tabular format.
<code>profile</code>	Collect and generate report for timeseries, cumulative and roofline data together.

4.5.2. Examples

4.5.2.1. Linux and FreeBSD

Table 4.4 **Linux and FreeBSD Examples#**

Command	Description
Collect IPC data from core 0 for the duration of 60 seconds	<code># ./AMDuProfPcm -m ipc -c core=0 -d 60 -o /tmp/pcmdata.csv</code>
Collect IPC/L3 metrics for CCX=0 for the duration of 60 seconds	<code># ./AMDuProfPcm -m ipc,l3 -c ccx=0 -d 60 -o /tmp/pcmdata.csv</code>

Command	Description
Collect only the memory bandwidth across all the UMCs for the duration of 60 seconds and save the output in /tmp/pcmdata.csv file	# ./AMDuProfPcm -m memory -a -d 60 -o /tmp/pcmdata.csv
Collect IPC data for 60 seconds from all the cores	# ./AMDuProfPcm -m ipc -a -d 60 -o /tmp/pcmdata.csv
Collect IPC data from core 0 and run the program in core 0	# ./AMDuProfPcm -m ipc -c core=0 -o /tmp/pcmdata.csv -- /usr/bin/taskset -c 0 <application>
Collect IPC data from cores 0-7 and run the application on cores 0-3	# ./AMDuProfPcm -m ipc -c core=0-7 -o /tmp/pcmdata.csv -- /usr/bin/taskset -c 0-3 <application>
Collect IPC and data l2 data from core 0 and report the cumulative (not timeseries) and run the program in core 0	# ./AMDuProfPcm -m ipc,l2 -c core=0 -o /tmp/pcmdata.csv -C -- /usr/bin/taskset -c 0 <application>
List the supported raw Core PMC events	# ./AMDuProfPcm -l
Print the name, description, and the available unit masks for the specified event	# ./AMDuProfPcm -z pmcx03
Compare two sessions	./AMDuProfPcm compare <output dir path 1>, <output dir path 2> Note Not applicable with --msr option.
Collect and generate roofline HTML report	./AMDuProfPcm roofline -O /tmp/ <application>

Command	Description
Collect default set of metrics, roofline and power data from all the cores of the system for 60 seconds and generate timeseries, cumulative, and roofline CSV and HTML reports	<code>./AMDuProfPcm profile -report-roofline --collect-power -O /tmp -d 60</code>
Collect ipc metrics from all the core in the system for 30 seconds and generate timeseries and cumulative CSV and HTML reports in the output directory	<code>./AMDuProfPcm profile -m ipc -O /tmp -a -d 30</code>

4.7. Interpreting Profile Data

4.7.1. Pipeline Utilization

On AMD *Zen4* and *Zen5*-based processors, AMDuProfPcm supports monitoring and reporting the pipeline utilization (pipeline_util) metrics. This feature provides pipeline_util metrics to visualize the bottlenecks in the CPU pipeline. Use the option `-m pipeline_util` to monitor and report the level-1 and level-2 top-down metrics.

Table 4.13 **Level-1 Metrics#**

Metric	Description
Total_Disp_Slots	Up to six instructions can be dispatched in one cycle.
SMT_Disp_contention	Unused dispatch slots as the other thread was selected.
Frontend_Bound	Dispatch slots that remained unused because the frontend did not supply appropriate instructions/ops.
Bad_Speculation	Dispatched operations that did not retire.

Metric	Description
Backend_Bound	Dispatch slots that remained unused because of backend stalls.
Retiring	Dispatch slots used by operations that retired.

Table 4.14 **Level-2 Metrics#**

Metric	Description
Frontend_Bound.Latency	Unused dispatch slots due to latency bottleneck in the frontend, such as Instruction Cache or ITLB misses.
Frontend_Bound.BW	Unused dispatch slots due to bandwidth bottleneck in the frontend, such as decode bandwidth or Op Cache fetch bandwidth.
Bad_Speculation.Mispredicts	Dispatched operations that were flushed due to branch mis-predicts.
Bad_Speculation.Pipeline_Restarts	Dispatched operations that were flushed due to pipeline restarts (resyncs).
Backend_Bound.Memory	Dispatched slots that remained unused because of stalls due to memory subsystem.
Retiring.Fastpath	Dispatch slots used by fastpath operations that retired.
Retiring.Microcode	Dispatch slots used by microcode operations that retired.

Due to multiplexing, the reported metrics may be inconsistent. For better results, use taskset to bind the monitored application to a specific set of cores and monitor only the cores on which the monitored application is running. Run the following command to collect the top-down metrics:

```
AMDuProfPcm -m pipeline_util --msr-A system -o /tmp/myapp-td.csv -- /usr/bin/taskset -c 0 myapp.exe
```

The `--msr` option requires root privileges. Run `sudo ./AMDPcmSetCapability.sh`, then open a new terminal tab or run without `--msr` option as noted here:

```
sudo AMDuProfPcm -m pipeline_util -c core=0 -A system -o /tmp/myapp-td.csv -- /usr/bin/ taskset -c 0 myapp.exe
```

Sample Top-Down Metrics report

Total_Dispatch_Slots	SMT_Dispatch	Frontend_Bound	Bad_Speculation	Backend_Bound	Retiring	IPC	Frontend_Bound.Latency	Frontend_Bound.BW	Bad_Speculation.Mispredicts	Bad_Spec	Backend_Bound.Memory	Backend_Bound.CPU	R
31528583352	0	0.18	0	33.19	66.39	3.85	0.18	0	0	0	13.15	20.04	
31801404234	0	0.19	0.96	32.79	66.98	3.95	0.19	0	0.91	0.05	13.13	19.66	
31876659852	0	0.18	0.22	32.73	67.53	3.95	0.18	0	0.2	0.02	13	19.73	
31889169876	0	0.18	0.08	32.54	66.96	4.01	0.19	0	0.08	0	12.94	19.6	
31914934404	0	0.19	0	32.76	66.92	3.97	0.19	0	0	0	12.99	19.77	
31991329650	0	0.19	0	39.84	59.33	3.53	0.21	0	0	0	14.25	25.59	
31851469458	0	0.19	0.22	59.47	40.21	2.36	0.22	0	0.04	0.18	16.48	42.99	
31949156628	0.01	0.19	0.21	59.3	40.39	2.33	0.22	0	0.04	0.17	16.43	42.87	
31817293530	0	0.19	0.6	59.15	40.39	2.38	0.22	0	0.1	0.5	16.52	42.64	
31832631192	0	5.95	18.46	43.48	32.79	1.84	5.44	0.5	18.39	0.07	20.61	22.87	
31912889772	0	10	31.77	31.46	26.5	1.48	9.21	0.79	31.75	0.02	19.36	12.1	
31883125182	0	10.08	31.78	31.71	26.5	1.47	9.24	0.84	31.76	0.02	19.51	12.2	
31993782744	0	10	30.81	31.72	26.47	1.45	9.15	0.85	30.79	0.02	19.37	12.35	
31858516134	0	10	31.56	31.83	26.32	1.45	9.15	0.85	31.54	0.02	19.6	12.23	
31928600622	0	9.95	31.78	31.85	26.49	1.43	9.03	0.92	31.76	0.02	19.62	12.24	
31802799444	0	9.98	32.24	31.71	26.48	1.45	9.13	0.85	32.22	0.02	19.66	12.05	
31827460368	0	5.4	17.59	38.25	40.03	2.25	5.04	0.37	17.56	0.03	21.73	16.52	
31937807754	0	0.14	0.05	44.44	54.97	3.22	0.17	0	0.03	0.02	22.78	21.66	
31865155098	0	0.15	0.07	44.79	54.97	3.22	0.17	0	0.04	0.03	22.97	21.83	
31977071160	0	0.15	0	44.79	54.89	3.18	0.17	0	0	0	22.8	22	

Figure 4.1 Sample Top-Down Metrics report#

Examples

Table 4.15 Top-Down Metrics Examples#

Task Description	Command
Timeseries monitoring of level-1 and level-2 top-down metrics (pipeline utilization) of a single- threaded program	# AMDuProfPcm -m pipeline_util -c core=1 -o /tmp/td.csv -- /usr/bin/taskset -c 1 /tmp/myapp.exe
Timeseries monitoring of level-1 and level-2 top-down metrics of a multi-threaded program running on all the cores:	# AMDuProfPcm -m pipeline_util -a -A system -o /tmp/td.csv -- /tmp/myapp.exe
Cumulative monitoring of level-1 and level-2 top-down metrics of a multi-threaded program running on all the cores	# AMDuProfPcm -m pipeline_util -a -A system -C -o /tmp/td.csv -- /tmp/myapp.exe

4.8. Virtualization Support

Profiling capabilities of AMDuProfPcm might be limited on a virtual machine. Check the following hardware and OS primitives provided by host or guest operating system to determine the level of support.

Run the command `AMDuProfCLI info - system` to obtain this information and look for the following sections.

Table 4.16 Virtualization Support System Information#

Item	Description
PERF Features Availability	Availability of Core, L3 and DF PMCs on the system. If any of the PMCs are unavailable, the corresponding metrics will not be supported. Ex. On guest VMs, DF PMCs are not accessible due to security reasons, due to which DF metrics such as memory, xgmi, pcie cannot be collected.
Hypervisor Info	Hypervisor vendor and support can be used to determine if the hypervisor is supported. Also determine the mode – host or guest. Usually the host has unrestricted access to all the PMCs.

4.9. Known Behavior - Issues Due to BIOS Settings

Following is the known behavior of L2 Hit/Miss from HWPF metrics based on the BIOS settings:

- AMDuProfPcm L2 Hit/Miss from HWPF metric doesn't collect any data when all following options are disabled in BIOS:
 - L1 Stream HW Prefetcher
 - L1 Stride Prefetcher
 - L1 Region Prefetcher
 - L2 Stream HW Prefetcher
 - L2 up/Down Prefetcher
- AMDuProfPcm L2 Hit/Miss from HWPF metric collects very less samples with the following BIOS settings:
 - L1 Stream HW Prefetcher: Disable
 - L1 Stride Prefetcher: Disable
 - L1 Region Prefetcher: Enable
 - L2 Stream HW Prefetcher: Disable
 - L2 up/Down Prefetcher: Disable

Roofline plots generated using AMDuProfModelling.py and saved as PDF might have improperly aligned labels for the plot lines.

4.10. Constraints and Limitations

Here is a list of constraints and limitations:

- Multiple instances of AMDuProfPcm can be launched in MSR mode (option `--msr`) with the force reset option (`-r`), doing so might cause undefined behavior.
- Sampling interval lower than 1 s (1000 ms) is not supported.
- MSR mode has significant overhead due to which the number of collected samples may not be equal to Profile duration/Sampling interval.
- In MSR mode, we can observe L3 Miss % to be > 100% when the number of L3 Accesses/Misses is low due to noise/inaccuracies.
- Undefined behavior if cores are turned off during a profile run.
- Heterogenous system configurations might cause undefined behavior.
- Hypervisors (from Cloud vendors) might restrict the guest from collecting some events due to security purposes, which leads to count values of these events being reported as 0.
- Limited support for Zen1 and Zen2 AMD processors.

4.11. Monitoring Without Root Privileges

On Linux, use the script `AMDPcmsetCapability.sh` to run the msr mode without root privilege. This option collects Core, L3, and DF PMC events on AMD Zen-based processors. The newer processors may require the latest kernel for Perf mode support.

Examples

Table 4.17 **Examples to Monitor Without Root Privileges#**

Task Description	Command
Cumulative reporting of IPC metrics at the end of the benchmark execution	<code>\$ AMDuProfPcm -m ipc -C -O /tmp -- /tmp/myapp.exe</code>
Cumulative reporting of IPC metrics at the end of the benchmark execution, aggregate metrics at system level	<code>\$ AMDuProfPcm -m ipc -C -A system -O /tmp -- /tmp/ myapp.exe</code>
Cumulative reporting of IPC metrics at the end of the benchmark execution, aggregate metrics per processor package	<code>\$ AMDuProfPcm -m ipc -C -A package -O /tmp -- /tmp/ myapp.exe</code>
Cumulative reporting of level-1 and level-2 top-down metrics (pipeline utilization)	<code>\$ AMDuProfPcm -m pipeline_util -C -A system -O /tmp -- /tmp/myapp.exe</code>
Timeseries monitoring of IPC of a benchmark, aggregate metrics at system level	<code>\$ AMDuProfPcm -m ipc -A system -O /tmp -- /tmp/ myapp.exe</code>
Timeseries monitoring of IPC of a benchmark, aggregate metrics per processor package	<code>\$ AMDuProfPcm -m ipc -A package -O /tmp -- /tmp/ myapp.exe</code>
Timeseries monitoring of IPC of a benchmark, system aggregate level	<code>\$ AMDuProfPcm -m ipc -O /tmp -- /tmp/myapp.exe</code>
Timeseries monitoring of level-1 and level-2 top-down metrics (pipeline utilization)	<code>\$ AMDuProfPcm -m pipeline_util -A system -O /tmp -- / tmp/myapp.exe</code>

Task Description	Command
Timeseries monitoring of memory bandwidth reporting at package and memory channels level	<code>\$ AMDuProfPcm -m memory -a -A system,package -O /tmp</code>

Performance Modelling using AMDuProfPcm

5. Performance Modelling using AMDuProfPcm

5.1. Classic Roofline Model

AMDuProfPcm provides basic roofline modeling that relates the application performance to memory traffic and floating point computational peaks. This is a visual performance model offering insights on improving the parallel software for floating point operations. This helps to characterize an application and identify whether a benchmark is memory or compute bound.

The tool monitors the memory traffic and floating point operations when the profiled application is running. Also, it computes the Arithmetic Intensity that is **Operations per byte of DRAM traffic [FLOPS/BYTE]**. The roofline chart is plotted as:

- **X-axis:** (AI) Arithmetic Intensity (FLOPS/byte) in logarithmic scale
- **Y-axis:** Throughput (GFLOPS/sec) in logarithmic scale
- **Horizontal line:** Peak theoretical floating-point performance of the system (HW Limit).
- **Diagonal line:** Peak memory performance. This line is plotted using the formula $\text{Throughput} = \min(\text{peak theoretical GFLOPS/Second}, \text{Peak theoretical Memory Bandwidth} * \text{AI})$.

By default, the tool plots horizontal rooflines for:

- Single Precision Floating Point Peak (SP FP Peak)
- Double Precision Floating Point Peak (DP FP Peak)

The options available to plot the max peak horizontal (computational) peak rooflines are:

- Single precision noSIMD and noFMA
- Double precision noSIMD and noFMA

5.2. Command Line Options

Generating the roofline chart of an application:

1. Collect and generate roofline HTML plot using AMDuProfPcm.

```
$ AMDuProfPcm roofline -O /tmp -- /tmp/myapp.exe
```

An output directory is created in the specified dir (/tmp) in the format AMDuProfPcm-Roofline-`<date>-<time>` which contains the HTML report (report.html). Open the HTML report and view the roofline graph located in the “ ” tab.

On AMD *Zen 4* and later processors, if the Linux kernel version doesn't support accessing DF counters, use the following command with root privilege.

```
$ AMDuProfPcm roofline --msr -O /tmp/ -- /tmp/myapp.exe
```

2. To generate the PDF roofline chart, run the following command (to be deprecated).

```
$ AMDuProfModelling.py -i /tmp/myapp-roofline.csv -o /tmp/ --memspeed 3200 -a myapp
```

The roofline chart is saved in the file /tmp/AMDuProf_roofline-yyyy-mm-dd-hhmmss.pdf.

A few pointers for generating the roofline chart:

- Use option `--read-smbios` to get memory speed and the number of memory channels from the SMBIOS table. This requires root privileges.
- If the AMDuProfPcm is launched with non-root privilege, While collecting the data, specify the DRAM speed using `-memspeed` option with the AMDuProfModelling. py script. You can use `dmidecode` or `lshw` command to get the memory speed.
- To plot additional computational horizontal peaks line, use the following options with AMDuProfModelling. py script:
 - `--sp-roofs`: Plot maximum peak roof for single-precision noSIMD and noFMA
 - `--dp-roofs`: Plot maximum peak roof for double-precision noSIMD and noFMA

Example

```
$ AMDuProfModelling.py -i /tmp/myapp-roofline.csv -o /tmp/ --memspeed 3200 -a myapp -dp-roofs
```

- Use `-a <appname>` option with AMDuProfModelling. py script to specify the application name to print in the graph chart.
- As this tool uses the maximum theoretical peaks for memory traffic and floating-point performance, you can use benchmarks such as STREAM to get the peak memory bandwidth and HPL or GEMM for peak FLOPS. Those scores can be used to plot the roofline charts. Use the following options with AMDuProfModelling. py script:
 - `--stream <STREAM score>`
 - `--hpl <HPL score>`
 - `--gemm <SGEMM | DGEMM score>`

Sample Roofline Chart

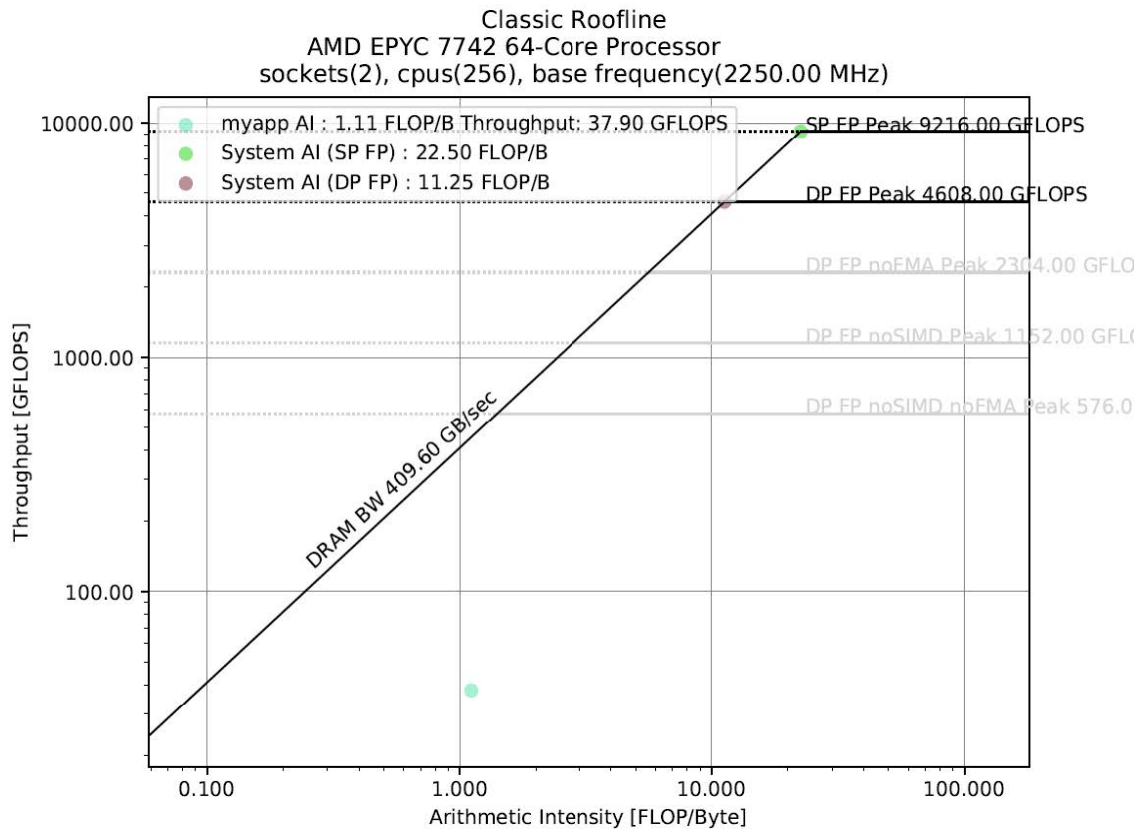


Figure 5.1 Sample Roofline Chart#